



HIGH PERFORMANCE COMPUTING FOR WEATHER AND CLIMATE

Exploring the Tradeoff Between Resolution and Floating-Point Precision in 2D Burgers' Equation

Authors

MAX KIMMLINGEN

MARCO MUCCIOLI

DAVID TSCHAN

SEPTEMBER 14, 2025

Contents

1 Introduction	2
2 Data & Methods.....	2
2.1 Test Case: 2D Burger's Equation	2
2.2 Numerical Setup	3
2.3 Parallelization Strategy	4
2.4 Benchmark and Tested Configurations.....	5
3 Results and Discussion.....	5
3.1 Precision-Resolution Tradeoff.....	6
3.2 l^2 Error and Power Spectral Density	7
3.3 Runtimes	8
4 Conclusion	9

1 Introduction

Weather and climate models simulate the complex physics of the climate systems by solving partial differential equations, such as the Navier-Stokes equations, on a grid. At the core of these models, numerical solutions rely heavily on stencil computations, which iteratively update field variables by applying fixed patterns to local neighborhoods of grid points.

Given the limited number of floating-point operations per memory access, stencil programs are typically memory-bound: their performance is constrained by memory bandwidth, i.e. how fast data can be transferred to and from memory, rather than raw computational speed. To improve performance, one strategy is to reduce memory transfers, for instance by lowering the floating-point precision used in calculations. Indeed, reduced-precision arithmetic (e.g., 16-bit instead of 32-bit) has already been explored as a means to improve performance while preserving adequate accuracy in specific modeling tasks (Klöwer et al. 2020).

While reducing precision can mitigate memory bottlenecks, another critical factor in model performance is grid resolution, which directly impacts the representation of physical processes. As a matter of fact, lowering the horizontal grid resolution decreases the total data volume processed. However, the scientific community is increasingly pushing for higher-resolution modeling, since finer grids allow more accurate representation of small-scale physical processes, such as convection and boundary layer dynamics (Schär et al. 2020).

This tradeoff between model resolution and floating-point precision is particularly relevant in the context of high-performance computing (HPC), as both significantly impact simulation accuracy and computational performance.

The complex interplay between these two factors raises a critical question: Given a fixed computational budget, is it more beneficial to increase model resolution or floating-point precision? We investigate this by exploring various precision–resolution combinations and comparing them against a high-resolution, high-precision benchmark. This will offer insights for future high-performance climate experiments. To this end, we implement the analysis in C++ for fast and scalable execution.

2 Data & Methods

2.1 Test Case: 2D Burger’s Equation

To investigate the tradeoff between floating-point precision and spatial resolution, we require a test case with the dynamics of both advection, which involves first-order spatial derivatives, and diffusion, consisting of second-order derivatives that induces shear. A purely diffusive equation lacks nonlinear features and offers limited insight into the benefits of increased resolution. Therefore, we adopt the *two-dimensional viscous Burger’s equation*, which includes both terms. The Burgers’ equation finds applications across various fields, including gas dynamics, fluid mechanics, shallow water theory, and traffic flow, where it models phenomena involving the balance of inertial and dissipative forces, leading to behaviors like wave propagation and shock formation ([Burgers’ equation \(Wikipedia\)](#)). In our setup, we solve the viscous form of the Burgers’ equation (eq. (1)):

$$\frac{d\mathbf{u}}{dt} = \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \cdot \mathbf{u} = \nu \Delta \mathbf{u} \quad (1)$$

in two spatial dimensions, with $\mathbf{u} : [0, T] \times \Omega \rightarrow \mathbb{R}^2, t \in [0, T] \times (x, y) \in \Omega \subseteq \mathbb{R}^2 \mapsto$

$(u, v) \in \mathbb{R}^2$ denoting velocity and $\nu \in \mathbb{R}_+$ representing kinematic viscosity, on the unit square $\Omega = [0, 1] \times [0, 1]$, for some final time $T \in \mathbb{R}$. Decomposed into its components, the equation reads

$$\begin{cases} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \\ \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right) \end{cases} \quad (2)$$

The equations are solved on horizontal slices, assuming no vertical extent of the domain. The inclusion of viscosity smooths the solution and prevents the formation of discontinuities, while keeping nonlinear advection dynamics.

2.2 Numerical Setup

To solve the Burger's equation numerically, we use an upwind scheme for spatial discretization in combination with an explicit Forward Euler step for time integration. Let u_{ij}^n and v_{ij}^n denote the spatiotemporal discretization of the velocity components on the domain $[0, T] \times \Omega$. Then the time is indicated by the timestep t with $\Delta t = T/N_t$ and $t = n\Delta t$ where $0 \leq n \leq N_t$ with N_t and n indicating the number of timestep the domain is discretised into and the time index, respectively. The spatial domain is decomposed into $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$, respectively, in the same way, so that $x_i = i\Delta x$ and $y_j = j\Delta y$ with $0 \leq i \leq N_x$ and $1 \leq j \leq N_y$, and $\mathbf{u}(x_i, y_j, t) = (u_{ij}^n, v_{ij}^n)$. Denoting with $(\bullet)_t$, $(\bullet)_x$ and $(\bullet)_y$ time-, x-direction- and y-direction derivatives, the Forward Euler timestep reads then:

$$(u_{ij}^n)_t \approx \frac{u_{ij}^{n+1} - u_{ij}^n}{\Delta t} \quad (3)$$

$$(v_{ij}^n)_t \approx \frac{v_{ij}^{n+1} - v_{ij}^n}{\Delta t} \quad (4)$$

In the upwind approach, first-order spatial derivatives are approximated using finite differences that take into account the direction of the flow, using values from upstream, where the wave is coming from. Consequently, first-order derivatives are approximated as:

$$(u_{ij}^n)_x \approx \begin{cases} \frac{u_{ij}^n - u_{i-1,j}^n}{\Delta x} & u_{ij}^n > 0 \\ \frac{u_{i+1,j}^n - u_{ij}^n}{\Delta x} & u_{ij}^n \leq 0 \end{cases} \quad (5)$$

$$(v_{ij}^n)_x \approx \begin{cases} \frac{v_{ij}^n - v_{i-1,j}^n}{\Delta x} & u_{ij}^n > 0 \\ \frac{v_{i+1,j}^n - v_{ij}^n}{\Delta x} & u_{ij}^n \leq 0 \end{cases} \quad (6)$$

$$(u_{ij}^n)_y \approx \begin{cases} \frac{u_{ij}^n - u_{i,j-1}^n}{\Delta y} & v_{ij}^n > 0 \\ \frac{u_{i,j+1}^n - u_{ij}^n}{\Delta y} & v_{ij}^n \leq 0 \end{cases} \quad (7)$$

$$(v_{ij}^n)_y \approx \begin{cases} \frac{v_{ij}^n - v_{i,j-1}^n}{\Delta y} & v_{ij}^n > 0 \\ \frac{v_{i,j+1}^n - v_{ij}^n}{\Delta y} & v_{ij}^n \leq 0 \end{cases} \quad (8)$$

Though more dissipative than central-difference schemes, the method is generally more robust and stable, particularly in the presence of nonlinearities, such as steep gradients and shock-like structures. Second-order derivatives in the Laplacian are discretized using central

differences, which on a uniform grid ($\Delta x = \Delta y$) yields the standard 5-point stencil:

$$\nabla^2 u_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2}, \quad (9)$$

$$\nabla^2 v_{i,j} \approx \frac{v_{i+1,j} + v_{i-1,j} + v_{i,j+1} + v_{i,j-1} - 4v_{i,j}}{\Delta x^2}. \quad (10)$$

Contrasting to upwind schemes, central-difference schemes are easier to implement and more accurate for smooth solutions, but can lead to artificial oscillations (Swanson et al. 1992).

The initial conditions for the velocity fields are chosen with the idea to represent a shear field where two flows flowing in opposite direction meet, a common pattern in weather and climate modeling observed in jet streams or oceanic currents. Specifically, they're defined as:

$$u_{ij}^0 = \cos\left(\frac{j\pi}{N_y}\right) \quad 0 \leq i \leq N_x \quad 0 \leq j \leq N_y \quad (11)$$

$$v_{ij}^0 = 0.5 - X_{ij} \quad X_{ij} \sim \mathcal{U}_{[0,1]} \quad (12)$$

The v component is initialized with small random fluctuations of 0.5 around the mean of 1, introducing noise that should help trigger real-world turbulence. This noise is likely¹ uniformly distributed, indicated by $X_{ij} \sim \mathcal{U}_{[0,1]}$. Figure 1a shows the initial u and v fields, along with corresponding velocity magnitude.

The boundary conditions are chosen to be periodic: anything leaving the domain on the left re-enters from the right, and anything leaving at the top re-enters from the bottom. Periodic boundaries ensure that the solution evolves without artificial influences from the domain edges. This is particularly suitable for studying nonlinear phenomena in the Burgers' equation, as it allows features to propagate freely without reflection or damping. Moreover, periodic boundaries maintain homogeneity and isotropy, which are essential for analyzing turbulence and energy transfer. In contrast, Dirichlet or Neumann conditions impose fixed values or gradients at the edges, potentially interfering with the internal dynamics and leading to artificial behavior.

2.3 Parallelization Strategy

The parallelization strategy for solving the Burger's equation is based on shared-memory parallelism, using OpenMP. In this approach, multiple threads access and operate on a common memory space within a single node, which allows execution both on high-performance clusters (scaling the numbers of cores) or on single-node machines.

The primary computational workload consist of three-nested loops iterating over the 3D grid. The parallelization focuses on the middle (j) and outer (k) loops, as their iterations are independent and involve only read access to shared variables. This makes them ideal candidates for parallel execution, allowing multiple threads to process different rows of the grid without leading to race conditions. The `collapse` clause is used to merge the nested loops into a single iteration space, improving the workload distribution. Although the innermost (i) loop could technically be parallelized, we chose not to parallelize the entire loop structure

¹To implement this boundary conditions, the command `std::rand()` was used, which is generating pseudo-random numbers — it is assumed here that the probability to receive any number within its range is equal, hence the random numbers are uniformly distributed.

since the additional performance gain did not justify the overhead introduced.

OpenMP was chosen over MPI because the code is designed to run on a single node with shared memory. Unlike MPI, which requires explicit domain decomposition and introduces communication overhead, OpenMP allows for a more straightforward and effective implementation by adding parallel directives to the loops.

2.4 Benchmark and Tested Configurations

The simulations were carried out on the ALPS infrastructure of the Swiss National Supercomputing Center, which uses NVIDIA Grace-Hopper processors with 72 cores. Thus each simulations was run in parallel on 72 cores. All simulations ran from 0 s to 1 s. The timestep $\Delta t = \min(\tau_D, \tau_A)$ is chosen such that it accounts for characteristic diffusion and advection times τ_D and τ_A . By default, we set $\Delta x = \Delta y$ in this study, that is $N_x = N_y$, so that the characteristic times become:

$$\tau_D \approx \frac{3\Delta x^2}{16\nu} \quad (13)$$

$$\tau_A \approx 0.4 \frac{\Delta x}{\text{Re}^* \nu} \quad (14)$$

Note here that $\text{Re}^* = 1 \times 10^4$ is assumed by default so that the flow is turbulent.

As no analytical solution exists for the 2D Burgers' equation², we defined a benchmark reference case to evaluate the different simulations. This benchmark consists of a high-resolution simulation with $N_x = N_y = 1024$ with long double precision.

The precision-resolution configurations investigated and the benchmark configuration are listed in Table 1. Although we did not limit the runtime, the results from the tested configurations provide insights into the general dependency of the accuracy on precision and resolution.

We attempted an implementation with half precision (`_fp16`) but it was excluded from the results. Although compilation and execution were possible, the precision conversion performed during regular compilation produced output files with mixed byte sizes, making the simulations unreliable. An alternative option, `_Float16`, would have been preferable; however, compilation with this data type was not successful despite extensive debugging.

3 Results and Discussion

The simulation results of the benchmark run are shown in Figure 1. While the top panel (Fig. 1a) displays the initial conditions of the velocity fields, the bottom one (Fig. 1b) shows the state of both components after 0.99 s. The diffusive nature of the Burger's equation (due to its viscous term) is visible in the bottom central panel, which highlights the state of the v field after 0.99 s. Compared to the initial state, the v component at 0.99 s is smoothed out due to the advective-diffusive process. The bottom left panel, displaying the final u field, indicates the presence of turbulence from shear instability. Together, these results suggest that the simulations have captured the turbulent dynamics arising from the Burgers' equations. Our results then provide insights into a possible precision-resolution tradeoff in climate models, where turbulence plays a large role.

²The 1D viscous Burgers' equation admits an analytical solution using the Cole-Hopf transformation.

$N_x = N_y$	Δt	Type	Runtime
<i>Benchmarking Run</i>			
1024	3.91×10^{-4} s	long double	281.98 s
<i>Other runs</i>			
128	3.14×10^{-3} s	long double	3.37 s
		double	2.86 s
		float	2.83 s
256	1.57×10^{-3} s	long double	8.06 s
		double	5.82 s
		float	5.79 s
512	7.83×10^{-4} s	long double	41.95 s
		double	20.89 s
		float	14.06 s

Table 1: Overview over all simulation configurations. All simulations were run with $T = 1$ s $\nu = 1 \times 10^{-4}$ and $\text{Re}^* = 1 \times 10^4$.

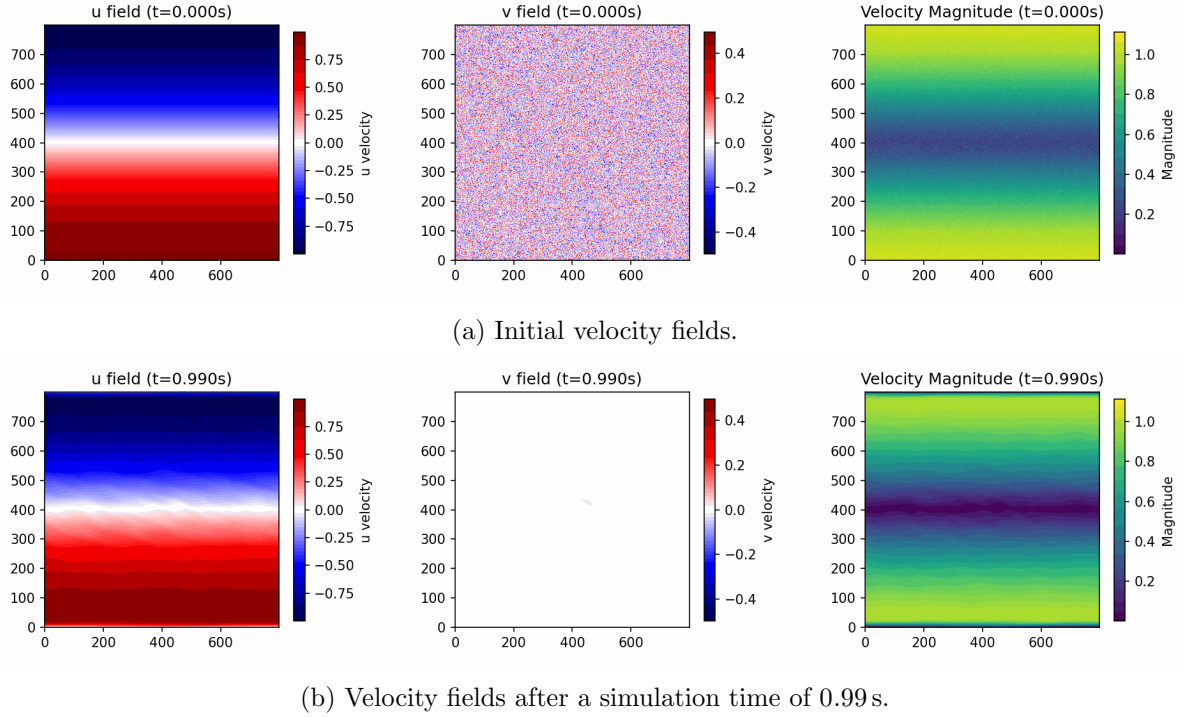


Figure 1: Benchmark simulation output showing the initial condition in the upper panel (1a), and the state of the domain after a simulation time of 0.99 s in the lower one (1b).

3.1 Precision-Resolution Tradeoff

Figure 2 illustrates the evolution of the u field and its vertical cross section for three different grid resolutions: $N_x = N_y = 128$ (a), $N_x = N_y = 256$ (b), and $N_x = N_y = 512$ (c). In each subfigure, the left panels show snapshots of the solution in long double precision at 0 s (initial condition), 0.5 s, and 0.1 s, whereas the right panel shows the cross section along the dotted

line (seen in left panel) at the same times and for all precisions. Note that only one line is visible because they all overlap.

Generally, across all resolutions, between the different precisions, there is no visible difference in the velocity fields (not shown). This feature also appears in the right panels of Figures 2 a, b, and c, where the plotted lines overlap for the three precisions. For our setup, floating-point precision appears to play a secondary role in achieving a high-accuracy solution.

In contrast, the subfigures clearly show differences among the tested resolutions at a given time step (except for the initial condition). With increasing resolution, the visible “waviness” (see the white regions for example of the left panels) decreases in both amplitude and wavelength. This is consistent with the intuitive expectation that higher resolution allows smaller wavelengths to be resolved. It also suggests that more energy is distributed toward these smaller scales. Consequently, we expect to observe clear differences in the error statistics across the different resolutions (see Section 3.2).

3.2 l^2 Error and Power Spectral Density

A more quantitative view on differences across numerical precisions is presented in Figure 3. It shows the evolution of two complementary error metrics we used to quantify the accuracy of the simulations relative to the benchmark run: the l^2 error and the error in the Power Spectral Density (PSD).

The l^2 error measures the root mean square difference between the simulated velocity fields and the benchmark, providing grid-point-wise measure of deviation. Formally, it is defined as:

$$l^2(u^{\text{sim}}, u^{\text{ref}}) = \sqrt{\frac{1}{N} \sum_{i=1}^N (u_i^{\text{sim}} - u_i^{\text{ref}})^2}$$

where u_i^{sim} denotes the simulated field (interpolated to the benchmark resolution), u_i^{ref} the benchmark field, and N the number of grid points.

Conversely, the PSD error evaluates how well the distribution of energy across spatial scales is captured by comparing Fourier spectra of the simulated and benchmark fields. We calculate it as

$$\text{PSD_error}(u^{\text{sim}}, u^{\text{ref}}) = l^2(|\hat{u}^{\text{sim}}|^2, |\hat{u}^{\text{ref}}|^2)$$

where $\hat{\cdot}$ represents the Fourier transformation.

Figure 3 shows that the error evolution is primarily driven by resolution, as lines of the same colour, suggesting the same resolution, overlap while lines of different colours are separated from each other. As resolution decreases, the errors increase. For the two highest-resolution simulations, the l^2 error reaches a peak before gradually decreasing, likely due to dissipation driving the solution toward a more homogeneous field. The absence of such a peak in the lowest-resolution simulation remains unexplained. In contrast, no peaks are observed in the PSD error, indicating that the differences between the tested configurations and the benchmark arise mainly from the representation of smaller wavelengths.

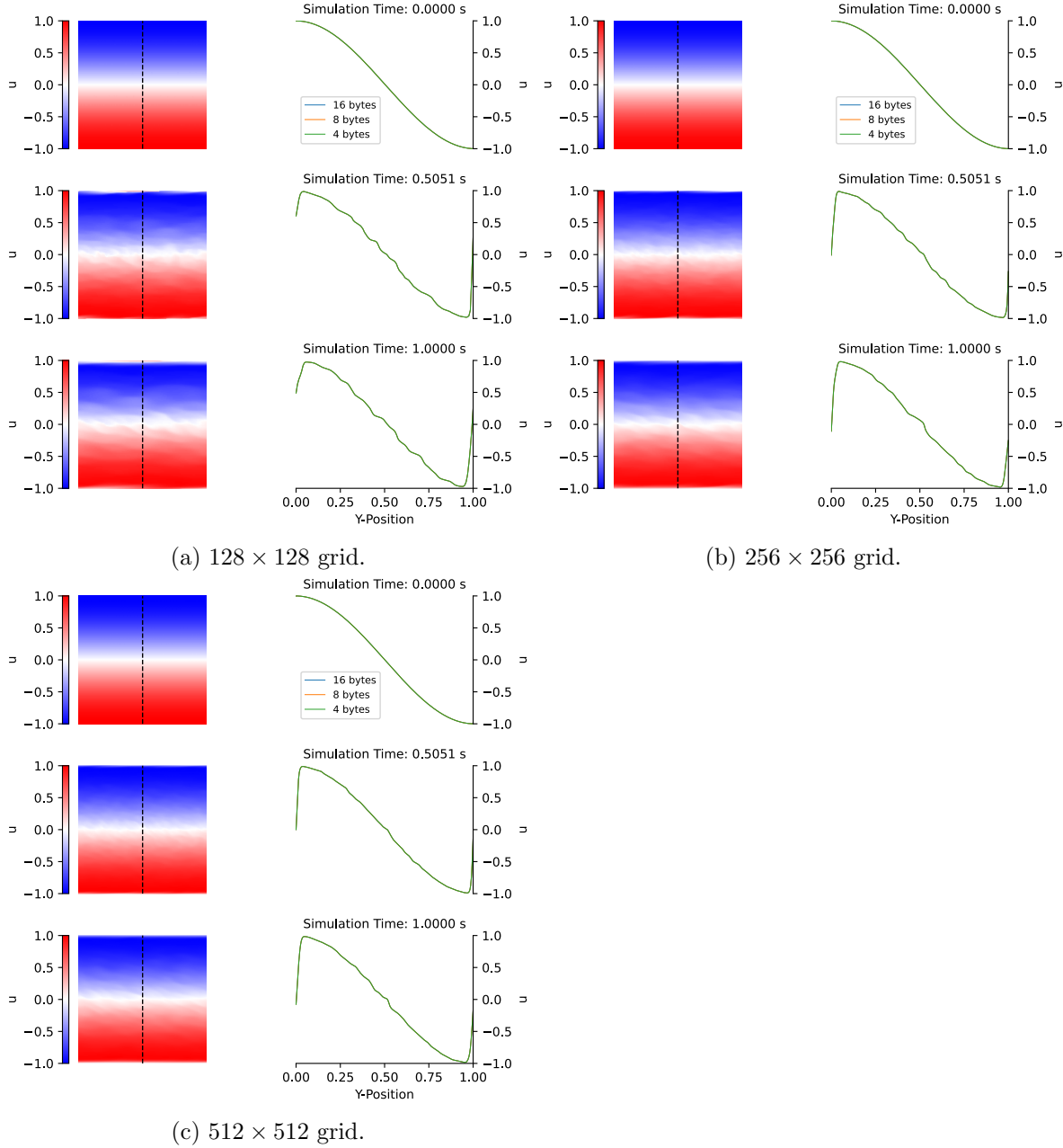


Figure 2: u -field in long double precision and cross section across the u -field in all precisions for 3 configurations and 3 time steps. Times shown: 0 s (top), 0.5 s (middle), 1 s (bottom). The black dotted line in the left panels indicates the cross section along which the profiles were computed. Label 4 stands for float, 8 for double and 16 for long double precision.

3.3 Runtimes

Table 1 displays the runtimes for all simulation configurations (see last column). For a purely memory-bound problem, a doubling of the precision should double the runtime, while a doubling of the resolution should multiply the runtime by a factor of 8. However, this is not observed here, potentially due to the computational overhead. For the 128×128 and 256×256 simulations, double and float precision exhibit roughly the same runtimes, whereas at 512×512 resolution, this is no longer the case. Although robust conclusions cannot be drawn from these results, they suggest that the rule described above holds more consistently at higher resolu-

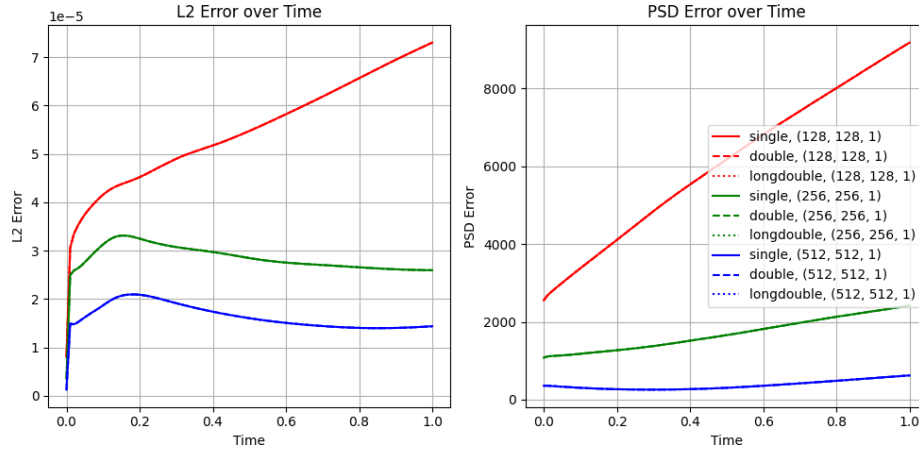


Figure 3: l^2 and PSD error for all configurations. Note: all lines of the same color overlap.

tions.

Despite the difficulty of predicting runtime from changes in resolution and precision, Figure 3 and Table 1 clearly indicate that resolution has a much stronger impact on accuracy than floating-point precision. For instance, changing the configuration from $(256 \times 256, \text{float})$ to $(512 \times 512, \text{float})$ multiplies the runtime by a factor of 2.41 while reducing the l^2 error by roughly 1.5. Conversely, switching from $(256 \times 256, \text{float})$ to $(256 \times 256, \text{long double})$ increases runtime by a factor of 1.4 but produces a negligible change in the l^2 error.

4 Conclusion

This study investigated the tradeoff between floating-point precision and spatial resolution in a stencil-based solver of the 2D viscous Burgers' equation. The performed experiments indicate that, for the turbulent setup tested here ($\nu = 1 \times 10^{-4}$, $\text{Re}^* = 10^4$), spatial resolution has a far greater impact on solution accuracy than floating-point precision. In all cases, an increase of precision produced no discernible increase in accuracy, while an increase in resolution resulted in more accurate simulations.

Our findings suggest that, for model components characterised by similar turbulent dynamics, investing computational resources in higher spatial resolution rather than increased floating-point precision is likely to yield greater benefit. This is consistent with prior work in weather and climate modelling, which demonstrated that reduced-precision arithmetic can provide performance advantages in memory-bound stencil codes without compromising accuracy (Klöwer et al. 2020).

To better understand how this tradeoff behaves in a climate model, a next step would be to perform a similar experiment with a climate model in an idealised setting (e.g. dry atmosphere, no topography, simple shear flow) before moving to more realistic settings, which include the interactions between the model's dynamical core and the parameterisations. We expect similar results in these situations, but a detailed analysis could provide insights for model optimisation. One could use it to identify an ideal precision-resolution configuration for a given runtime constraint.

References

- Klöwer, M., P. D. Düben, and T. N. Palmer (2020). “Number formats, error mitigation, and scope for 16-bit arithmetics in weather and climate modeling analyzed with a shallow water model”. In: *Journal of Advances in Modeling Earth Systems* 12, e2020MS002246. DOI: [10.1029/2020MS002246](https://doi.org/10.1029/2020MS002246). URL: <https://doi.org/10.1029/2020MS002246>.
- Schär, Christoph et al. (2020). “Kilometer-Scale Climate Models: Prospects and Challenges”. In: *Bulletin of the American Meteorological Society* 101, E567–E587. DOI: [10.1175/BAMS-D-18-0167.1](https://doi.org/10.1175/BAMS-D-18-0167.1). URL: <https://doi.org/10.1175/BAMS-D-18-0167.1>.
- Swanson, Roy C. and Eli Turkel (1992). “On central-difference and upwind schemes”. In: *Journal of Computational Physics* 101.2, pp. 292–306. DOI: [10.1016/0021-9991\(92\)90038-K](https://doi.org/10.1016/0021-9991(92)90038-K).